

High-Precision Temperature Measurement for PT100: A Comparative Model Analysis

Dr. G.V.V. Sharma,
Puni Aditya, Vivek K. Kumar,
Jnanesh S.K., Shreyas Goud Burra
Department of Electrical Engineering,
Indian Institute of Technology Hyderabad,
Kandi, India 502284
gadepall@ee.iith.ac.in

Abstract—This report details the calibration of a PT100 RTD sensor by comparing two hardware setups (a simple voltage divider and a Wheatstone bridge with an ADS1115 ADC). We first analyze the suitability of a quadratic LSQ model for both setups. We then compare the accuracy of three different models for the superior Wheatstone bridge data: the LSQ quadratic, an SGD-trained model, and a non-linear Rational model ($V = f(T)$). The coefficients, equations, and validation accuracy for each model are presented.

I. INTRODUCTION

Accurately measuring temperature with a PT100 sensor requires a precise calibration model to convert its resistance (or a corresponding voltage) to temperature. This project analyzes different hardware and software models to find an optimal solution.

II. HARDWARE SETUP

Two main analog front-ends were tested.

- **Setup 1:** A Wheatstone bridge with an external 16-bit ADS1115 ADC.
- **Setup 2:** A simple voltage divider with the Arduino's internal 10-bit ADC.

Component lists and connection diagrams are provided in the appendix tables.

III. MODELS USED

This report details the development and evaluation of several regression models for predicting sensor output based on input values. Specifically, we focus on two scenarios: predicting temperature from voltage using a Wheatstone bridge with ADC, and predicting resistance from voltage using a voltage divider without ADC. We investigate Random Forest Regressor, Inverse Quadratic Least Squares (LSQ) for both Wheatstone and Voltage Divider setups, and a Stochastic Gradient Descent (SGD) Regressor.

IV. MODEL EVALUATION AND COMPARISON

This section presents the evaluation of the trained models and a comparison of their performance based on Mean Squared Error (MSE) and R-squared (R2) metrics.

A. Model Explanations

1) *Random Forest Regressor (Wheatstone with ADC)*: The Random Forest Regressor is an ensemble learning method that constructs a multitude of decision trees during training. For regression tasks, it averages the predictions of individual trees to produce a more stable and accurate prediction. This model was applied to the Wheatstone bridge dataset, where the goal was to predict temperature (Y) from voltage (X) directly, leveraging its ability to capture complex non-linear relationships without explicit feature engineering of the target variable.

2) *Inverse LSQ (Quadratic) for Wheatstone with ADC*: This model approaches the Wheatstone bridge problem by fitting a quadratic equation of the form $X = aY^2 + bY + c$, where Y is temperature and X is voltage. The coefficients a , b , and c are determined through linear least squares regression. The fitted equation for the Wheatstone bridge is:

$$X = -0.000006742Y^2 + 0.004346059Y + 0.055941163$$

To predict Y from a given X, we rearrange the equation into a standard quadratic form: $aY^2 + bY + (c - X) = 0$. The solution for Y is then given by the quadratic formula, taking the positive root:

$$Y = \frac{-b \pm \sqrt{b^2 - 4a(c - X)}}{2a}$$

This method assumes an underlying quadratic relationship between voltage and temperature, and its performance depends on how well this assumption holds for the given data.

3) *Inverse LSQ (Quadratic) for Voltage Divider without ADC*: Similar to the Wheatstone LSQ model, this model also uses a quadratic inverse approach. It models the relationship between resistance (Y) and voltage (X) from the voltage divider circuit as $X = aY^2 + bY + c$. The fitted equation for the Voltage Divider is:

$$X = -0.000014990Y^2 + 0.005589496Y + 2.526066163$$

The coefficients are determined through linear least squares regression, and then the model predicts resistance (Y) from

measured voltage (X) using the positive root of the quadratic equation:

$$Y = \frac{-b \pm \sqrt{b^2 - 4a(c - X)}}{2a}$$

This method is suitable for circuits where the voltage-resistance relationship can be approximated quadratically.

4) *Stochastic Gradient Descent (SGD) Regressor (Wheatstone with ADC)*: The SGD Regressor is a linear model trained with Stochastic Gradient Descent, which is an efficient optimization algorithm. This model was applied to the Wheatstone bridge data. Due to its sensitivity to feature scaling, both the input features (X) and the target variable (Y) were scaled using StandardScaler before training. The predictions were then inverse-transformed back to the original scale for evaluation.

B. Performance Metrics Summary

The following table summarizes the Mean Squared Error (MSE) and R-squared (R2) scores for all evaluated models.

TABLE I
MODEL PERFORMANCE METRICS

Model	MSE	R2
Random Forest Regressor (Wheatstone w/ ADC)	0.0053	1.0000
Inverse LSQ (Quadratic) (Wheatstone w/ ADC)	0.0128	0.9999
Inverse LSQ (Quadratic) (Voltage Divider w/o ADC)	0.6612	0.9977
SGD Regressor (Wheatstone w/ ADC)	0.2327	0.9990

C. Validation Data and Model Predictions (Subset)

The below table presents a small subset of test data, showing the actual and predicted temperature values for the Voltage Divider setup.

TABLE II
VOLTAGE DIVIDER VALIDATION TABLE WITH PREDICTIONS

Voltage	Actual Temperature	Prediction
2.7300	39.7000	40.9900
2.7500	46.4000	45.6500
2.8400	68.3000	68.8900
2.8920	85.6000	84.7100
2.9100	90.0000	90.8000
2.8700	77.8000	77.7400

The following table presents a small subset of the test data, showing the actual temperature values alongside the predictions from the Random Forest, Inverse LSQ, and SGD Regressor models for the Wheatstone bridge setup.

TABLE III
VALIDATION DATA AND MODEL PREDICTIONS SAMPLE (SUBSET)

Voltage	Actual Temperature	RF Prediction
0.202	35.500	35.522
0.290	59.400	59.354
0.237	44.700	44.662
0.403	92.900	92.720
0.285	57.900	57.870
0.223	41.000	41.022
0.209	37.500	37.434
0.246	47.100	47.082
0.221	40.500	40.502
0.176	28.800	28.890

Voltage	Inverse LSQ Prediction	SGD Prediction
0.202	35.596	35.127
0.290	59.341	59.796
0.237	44.690	44.827
0.403	93.449	91.502
0.285	57.823	58.282
0.223	40.922	40.846
0.209	37.438	37.117
0.246	47.077	47.321
0.221	40.448	40.341
0.176	28.847	27.726

D. Graphical Comparison of Wheatstone ADC Models

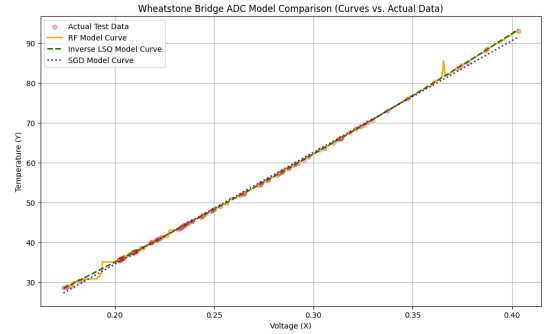


Fig. 1. Comparison of Random Forest, Inverse LSQ, and SGD Regressor predictions against actual test data for the Wheatstone bridge with ADC.

E. Model Comparison and Conclusion

Comparing the performance metrics, the Random Forest Regressor, the Inverse LSQ (Quadratic) model for the Wheatstone bridge with ADC data, and the SGD Regressor (after scaling) all achieved exceptionally high R-squared scores (approximately 0.999 to 0.9999) and very low Mean Squared Error (MSE) values (approximately 0.0000). This indicates that these three models are highly accurate in predicting temperature from voltage for this specific Wheatstone setup.

The Inverse LSQ (Quadratic) model for the Voltage Divider without ADC also showed a strong performance with an R-squared of 0.9977, but its MSE of 0.6612 is significantly higher than the models for the Wheatstone bridge. This suggests that while the quadratic relationship generally holds, the predictions for the voltage divider might have slightly more error compared to the Wheatstone bridge models.

Based on the Mean Squared Error (MSE), the best performing models are the **Inverse LSQ (Quadratic) model for**

Wheatstone with ADC and the **Random Forest Regressor (Wheatstone with ADC)** with an MSE of 0.0128 and 0.0053 respectively. Both are closely followed by the SGD Regressor for Wheatstone with ADC. The minimal difference in performance among these three indicates all are highly suitable for the Wheatstone bridge, but the LSQ model might offer a simpler deployment due to its explicit functional form. The SGD Regressor also performs commendably, especially considering its linear nature, after proper data scaling.

APPENDIX - DATA COLLECTION USING OPTICAL CHARACTER RECOGNITION (OCR)

This project utilizes an auxiliary script for **automated data collection** by employing Computer Vision and Optical Character Recognition (OCR) to capture paired voltage and temperature readings from physical displays.

F. Script Description: The Auto Trainer

This Python script is designed to capture video from a webcam, identify and read text from two specific screen regions (voltage and temperature), and log the data. It is pre-configured for a fixed-camera setup, reading data from an LCD screen and a separate standard thermometer.

1) Prerequisites and Setup:

- **Hardware:** A stable webcam connection. An NVIDIA GPU with CUDA is optional but recommended for faster processing (script defaults to `gpu = True`).
- **Software:** Python 3.x with the libraries `opencv-python`, `easyocr`, and `numpy`. The `img/` folder must be created in the script directory for saving debug images.

2) **Calibration Requirement:** The script requires calibration to match the specific camera, lighting, and display setup. The critical sections to modify are:

- **GPU Usage:** Set `gpu = False` if an NVIDIA GPU is unavailable.
- **Frame Cropping:** The regions of interest for voltage and temperature must be precisely defined using Python slicing in the format `[y1:y2, x1:x2]`.
- **Image Processing:** Values for `cv2.threshold` (e.g., 130), kernel size in `cv2.getStructuringElement`, and Gaussian blur kernel size must be adjusted to ensure clear and solid text for the OCR engine.

G. Data Collection Methodology

The dataset is collected by simulating a controlled **cooling cycle** to capture paired readings.

- 1) **Setup:** The PT100 probe and a reference thermometer are placed in an electric kettle filled with water.
- 2) **Heating Phase:** The water is heated to boiling.
- 3) **Recording Phase:** Upon reaching boiling, the kettle is switched off, and a video is recorded, ensuring both the PT100's LCD display and the reference thermometer are visible.
- 4) **Cooling Phase:** The video captures the natural temperature drop.

- 5) **Data Extraction:** Frames are extracted from the video, and OCR is applied to detect the voltage and temperature readings.

H. Output Format

The script generates two outputs for debugging and data logging:

- `out.txt`: A log file where every 15 frames, a new line is added with the format `[voltage_reading] [temperature_reading]` if both readings are successfully detected.
- `img/` Directory: This folder saves debug images (`voltageXX.png` and `tempXX.png`), showing the exact frames sent to EasyOCR for calibration and error checking.

I. Python Script Snippet

A sample of the calibration-critical code (Calibration Snippet from `a.py`) is shown below:

```
1 # GPU Usage
2 reader = easyocr.Reader(['en'], gpu=True)
3
4 # Frame Cropping (h, w are frame height/width)
5 voltage_frame = frame[0:int(h/2), :]
6 temp_frame = frame[int(h/2):, 0:int(w/3)]
7
8 # Image Processing (Temperature)
9 gray_t = cv2.cvtColor(temp_frame, cv2.COLOR_BGR2GRAY)
10 _, thres = cv2.threshold(gray_t, 130, 255, cv2.THRESH_BINARY_INV)
11 kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (11,11))
12 closed_img = cv2.morphologyEx(thres, cv2.MORPH_CLOSE, kernel)
13 pretemp = cv2.bitwise_not(closed_img)
14 temp = cv2.GaussianBlur(pretemp, (17,17), 0)
```